

Aufgabe 1

a)

```
.text
.globl main

main:
    addi x1, x0, 5      # S0: Fetch, S1: Decode, S8: ExecuteI, S7: ALUWB
    addi x2, x0, 10     # S0: Fetch, S1: Decode, S8: ExecuteI, S7: ALUWB
    add x3, x2, x1      # S0: Fetch, S1: Decode, S6: ExecuteR, S7: ALUWB
    sw x3, 0(x0)        # S0: Fetch, S1: Decode, S2: MemAdr, S5: MemWrite
    lw x4, 0(x0)        # S0: Fetch, S1: Decode, S2: MemAdr, S3: MemRead, S4:
MemWB
    beq x1, x1, label   # S0: Fetch, S1: Decode, S10: BEQ
    addi x0, x0, 0      # skipped by jump

label:
    jal x5, end         # S0: Fetch, S1: Decode, S15: Reset

end:
    ecall
```

b)

Die ALU wird in folgenden Zuständen verwendet:

- S0 Fetch: Hier wird die ALU verwendet, um den Programcounter zu erhöhen (PC + 4).
- S1 Decode: Hier berechnet die ALU die Summe des PC und des Immediate, um eine Branch-Zieladresse vorzuberechnen.
- S2 MemAdr: Hier berechnet die ALU die Speicheradresse, bestehend aus dem extended Immediate und dem Wert aus dem Register-File.
- S6 ExecuteR: Die ALU berechnet arithmetische/logische Operationen für R-Typ-Instruktionen (z.B.: add, sub, and, or)
- S8 ExecuteI: Die ALU berechnet arithmetische/logische Operationen für I-Typ-Instruktionen (z.B.: addi, andi, xori, ...)
- S10 BEQ: Die ALU vergleicht die beiden Operanden und das Ergebnis entscheidet, welcher Branch genommen wird.